Experiment 1 – Naïve Bayes Classifier

AIM

To implement a Naïve Bayes classifier on the **Breast Cancer Wisconsin (Diagnostic) dataset**, study the effect of different feature subsets on classification performance, examine the conditional-independence assumption, and compare classifier performance with and without Laplace smoothing.

WHAT?

Naïve Bayes is a supervised probabilistic classification algorithm based on Bayes' theorem. It predicts the class with the highest posterior probability while assuming that the features are conditionally independent given the class label.

WHEN?

It is used when a fast and simple probabilistic classifier is required, especially when the training data is limited or the features are approximately independent.

WHERE?

It is commonly used in spam filtering, sentiment analysis, document classification, medical diagnosis, recommendation systems, and fraud detection.

HOW?

The classifier calculates the prior probability of each class and the likelihood of the features given that class. These probabilities are combined using Bayes' theorem, and the class with the highest posterior probability is selected.

Dataset

Breast Cancer Wisconsin (Diagnostic) dataset

Samples = 569

Features = 30

Classes = 2

Classes = malignant, benign

The dataset is loaded directly using scikit-learn.

ALGORITHM

1. Import the required Python libraries.
2. Load the Breast Cancer Wisconsin dataset.
3. Separate the features X and target labels y.
4. Split the dataset into training and testing sets using a 75:25 ratio.
5. Create different feature sets:

- All 30 features
- 10 mean features
- 10 worst features
- Top 5 correlated features

6. Train a Gaussian Naïve Bayes classifier for each feature set.
7. Predict the test data.
8. Calculate accuracy, precision, and recall.
9. Convert continuous features into binary features using their training-set medians.
10. Train Bernoulli Naïve Bayes with:
    - No smoothing
    - Laplace smoothing
11. Repeat the smoothing experiment using only 20 training samples.
12. Calculate the average absolute correlation among the mean features.
13. Compare and interpret the results.

PROGRAM

```
import numpy as np
   X_test > medians
).astype(int)



for alpha, label in [
   (1e-10, "No smoothing"),
   (1.0, "Laplace smoothing")
]:


   clf = BernoulliNB(
      alpha=alpha
   ).fit(
      Xb_train,
      y_train
```

```
    )

    pred = clf.predict(Xb_test)

    print(
        label,
        accuracy_score(y_test, pred),
        precision_score(y_test, pred),
        recall_score(y_test, pred)
    )


# Part B2: Zero-frequency problem
# Small training sample (n=20)

X_small, _, y_small, _ = train_test_split(
    X_train,
    y_train,
    train_size=20,
    random_state=RANDOM_STATE,
    stratify=y_train
)

Xb_small = (
    X_small > medians
).astype(int)


for alpha, label in [
    (0.0, "alpha=0"),
    (1.0, "alpha=1")
```

```
]:

    clf = BernoulliNB(
        alpha=alpha
    ).fit(
        Xb_small,
        y_small
    )

    pred = clf.predict(Xb_test)

    print(
        label,
        accuracy_score(y_test, pred)
    )



# Part C: Feature-independence check

corr = np.corrcoef(
    X_train[:, 0:10],
    rowvar=False
)

avg_abs_corr = (
    (np.sum(np.abs(corr)) - 10)
    / (10 * 10 - 10)
)

print(
    "Average |correlation| among mean features:",
```

avg_abs_corr

)

This is the **same experiment and code structure as the reference PDF**, including the four feature sets, Laplace comparison, small-sample zero-frequency test, and feature-correlation check.

OUTPUT

Dataset shape: (569, 30) Classes: ['malignant' 'benign']

--- Part A: GaussianNB performance across feature sets ---

All 30 features

Accuracy=0.9371

Precision=0.9263

Recall=0.9778

Mean features only (10)

Accuracy=0.9161

Precision=0.9239

Recall=0.9444

Worst features only (10)

Accuracy=0.9371

Precision=0.9451

Recall=0.9556

Top-5 correlated features

Accuracy=0.9091

Precision=0.8969

Recall=0.9667

--- Part B: BernoulliNB with vs without Laplace smoothing ---

No smoothing

Accuracy=0.9301

Precision=0.9762

Recall=0.9111

Laplace smoothing

Accuracy=0.9231

Precision=0.9759

Recall=0.9000

--- Part B2: Zero-frequency effect with a small training sample (n=20) ---

alpha=0

Accuracy=0.3706

alpha=1

Accuracy=0.9021

--- Part C ---

Average |correlation| among mean features = 0.4794

These are the outputs reported in the reference PDF.

RESULT

The Gaussian Naïve Bayes classifier achieved **93.71% accuracy, 92.63% precision, and 97.78% recall** using all 30 features. The worst-feature subset achieved the same accuracy with a higher precision of **94.51%**.

Experiment 2 – Maximum Likelihood Estimation vs Bayesian Estimation

AIM

To estimate the mean and variance of the **mean radius** feature of benign tumour samples from the Breast Cancer Wisconsin dataset using **Maximum Likelihood Estimation (MLE)** and Bayesian estimation, and study how sample size and prior assumptions affect the estimates.

WHAT?

MLE estimates parameter values using only the observed data, choosing the values that make the data most probable.

Bayesian estimation combines a prior belief about the parameter with the observed data to obtain a posterior estimate.

WHEN?

MLE is useful when sufficient data is available and reliable prior knowledge is not available. Bayesian estimation is useful when the sample is small or prior knowledge needs to be incorporated.

WHERE?

These methods are used in medical parameter estimation, clinical trials, A/B testing, signal processing, quality control, and financial risk modelling.

HOW?

MLE calculates the sample mean and variance directly from the data. Bayesian estimation combines the prior mean and variance with the observed sample using a precision-weighted update.

Dataset

The experiment uses the **mean radius** feature of the benign samples from the Breast Cancer Wisconsin dataset.

Number of benign samples = 357

Prior mean = 15.0

Prior variance = 4.0

The reference assumes a Normal prior:

N(15.0, 4.0)

ALGORITHM

1. Load the Breast Cancer Wisconsin dataset.
2. Extract the mean radius feature for benign samples.
3. Calculate the true population mean and variance.

4. Define the Bayesian prior mean and prior variance.
5. For different sample sizes:
    - Randomly select samples.
    - Calculate the MLE mean and variance.
    - Calculate the Bayesian posterior mean.
    - Calculate the error from the true population mean.
6. Repeat the sampling process 500 times for sample sizes 5 and 50.
7. Calculate the standard deviation of the MLE and Bayesian estimates.
8. Compare the estimates and their variability.

PROGRAM

```
import numpy as np


  n = len(sample)


  sample_mean = np.mean(sample)


  post_var = 1 / (
    1 / prior_var +
    n / known_var
  )


  post_mean = post_var * (
    prior_mean / prior_var +
    n * sample_mean / known_var
  )


  return post_mean, post_var



# Random generator
```

```
rng = np.random.default_rng(42)



# Compare different sample sizes
for n in [3, 5, 10, 30, 100, 357]:

    idx = rng.choice(
        len(feature),
        size=n,
        replace=False
    )

    sample = feature[idx]

    mu_mle, var_mle = mle_estimate(sample)

    mu_bayes, _ = bayesian_estimate(
        sample,
        prior_mean,
        prior_var,
        population_var
    )

    print(
        n,
        mu_mle,
        mu_bayes,
        var_mle,
        abs(mu_mle - true_mean),
        abs(mu_bayes - true_mean)
    )
```

```
# Variability over repeated sampling
for n in [5, 50]:

    mle_vals = []
    bayes_vals = []

    for _ in range(500):

        idx = rng.choice(
            len(feature),
            size=n,
            replace=False
        )

        sample = feature[idx]

        mle_vals.append(
            mle_estimate(sample)[0]
        )

        bayes_vals.append(
            bayesian_estimate(
                sample,
                prior_mean,
                prior_var,
                population_var
            )[0]
        )
```

```
    print(
        n,
        np.std(mle_vals),
        np.std(bayes_vals)
    )
```

The program follows the reference experiment, including sample sizes 3, 5, 10, 30, 100, 357 and the 500 repeated samples for variability analysis.

OUTPUT

Full population (benign, n=357):

mean=12.1465

var=3.1613

n | MLE mean | Bayes mean | MLE var | |MLE-true| | |Bayes-true|

3 | 12.6333 | 13.1268 | 0.5939 | 0.4868 | 0.9803

5 | 11.7198 | 12.1675 | 2.5923 | 0.4267 | 0.0210

10 | 12.9220 | 13.0742 | 2.2871 | 0.7755 | 0.9277

30 | 12.6691 | 12.7289 | 2.5328 | 0.5226 | 0.5824

100 | 12.1594 | 12.1817 | 3.0567 | 0.0129 | 0.0351

357 | 12.1465 | 12.1528 | 3.1613 | 0.0000 | 0.0063

Repeated-sampling variability:

n=5

MLE std = 0.8093

Bayes std = 0.6988

n=50

MLE std = 0.2232

Bayes std = 0.2197

These are the results reported in the reference PDF.

RESULT

For very small samples, both MLE and Bayesian estimates can deviate from the true population mean. However, the Bayesian estimate showed lower variability for n=5:

MLE standard deviation = 0.8093

Bayesian standard deviation = 0.6988

Experiment 3 – Expectation-Maximization (EM) Algorithm for a Mixture Model

AIM

To implement a Gaussian Mixture Model using the **Expectation-Maximization (EM) algorithm** on the Iris dataset and study how the number of mixture components and initialization method affect convergence speed, final log-likelihood, and local optima.

WHAT?

EM is an iterative algorithm used for models containing **hidden or latent variables**. It alternates between:

- **E-step:** calculates the probability that each data point belongs to each mixture component.
- **M-step:** updates the model parameters using those probabilities.

The process repeats until the log-likelihood converges.

WHEN?

EM is used when data is believed to come from multiple underlying groups that are not directly labelled, such as soft clustering and problems involving hidden variables.

WHERE?

It is used in customer segmentation, image segmentation, speech recognition, anomaly detection, and gene-expression clustering.

HOW?

The algorithm starts with initial mixture parameters, performs the E-step to calculate responsibilities, performs the M-step to update the parameters, and repeats until convergence. Different initialization methods and multiple restarts can be used to improve stability.

DATASET

The experiment uses the **Iris dataset**.

Samples = 150

Features used = Petal Length, Petal Width

True classes = 3 Iris species

The true species labels are retained only for evaluating clustering quality using the **Adjusted Rand Index (ARI)**.

ALGORITHM

1. Load the Iris dataset.
2. Select petal length and petal width.
3. Set the true species labels aside for evaluation.
4. Train Gaussian Mixture Models with k = 2, 3, 4, 5.
5. Record convergence, iterations, log-likelihood, BIC, and AIC.
6. Fix k = 3 and vary the random initialization seeds.
7. Calculate the ARI for each initialization.
8. Compare random and k-means++ initialization over 20 runs.
9. Test multiple restarts using n_init = 1, 5, 10.
10. Compare the final log-likelihoods and convergence behavior.

PROGRAM

```
import numpy as np


  gmm = GaussianMixture(
    n_components=3,
    init_params='random',
    random_state=s,
    max_iter=200,
    n_init=1
  ).fit(X)

  labels = gmm.predict(X)

  print(
    s,
    gmm.converged_,
```

```
        gmm.n_iter_,
        gmm.score(X) * len(X),
        adjusted_rand_score(
            y_true,
            labels
        )
    )


# Part C: Initialization strategy stability

for init_strategy in [
    'random',
    'k-means++'
]:

    lls = []

    for s in range(20):

        gmm = GaussianMixture(
            n_components=3,
            init_params=init_strategy,
            random_state=s,
            max_iter=200,
            n_init=1
        ).fit(X)

        lls.append(
            gmm.score(X) * len(X)
        )
```

```
    print(
        init_strategy,
        np.mean(lls),
        np.std(lls),
        max(lls),
        min(lls)
    )



# Part D: Multiple restarts


for n_init in [1, 5, 10]:


    gmm = GaussianMixture(
        n_components=3,
        init_params='random',
        random_state=42,
        max_iter=200,
        n_init=n_init
    ).fit(X)


    print(
        n_init,
        gmm.score(X) * len(X),
        gmm.n_iter_
    )
```

This follows the reference PDF's four experimental parts: varying k, varying random seeds, comparing initialization strategies, and testing multiple restarts.

OUTPUT

Part A – Number of Components

k=2 | converged=True | n_iter=3 | log_lik=-154.73 | BIC=364.58 | AIC=331.46

k=3 | converged=True | n_iter=17 | log_lik=-135.45 | BIC=356.08 | AIC=304.90

k=4 | converged=True | n_iter=9 | log_lik=-130.56 | BIC=376.36 | AIC=307.11

k=5 | converged=True | n_iter=25 | log_lik=-125.14 | BIC=395.58 | AIC=308.28

Part B – Random Initialization

seed=0 | n_iter=27 | log_lik=-134.21 | ARI=0.562

seed=1 | n_iter=21 | log_lik=-134.20 | ARI=0.559

seed=5 | n_iter=17 | log_lik=-134.20 | ARI=0.583

seed=6 | n_iter=14 | log_lik=-134.16 | ARI=0.611

seed=9 | n_iter=19 | log_lik=-134.24 | ARI=0.554

Log-likelihood spread across 10 seeds: 0.08

Part C – Initialization Stability

random

mean_ll=-138.39

std_ll=18.128

best_ll=-134.16

worst_ll=-217.40

k-means++

mean_ll=-135.51

std_ll=0.245

best_ll=-135.35

worst_ll=-136.10

Part D – Multiple Restarts

n_init=1 | best_log_likelihood=-134.24 | n_iter=16

n_init=5 | best_log_likelihood=-134.17 | n_iter=19

n_init=10 | best_log_likelihood=-134.16 | n_iter=18

These outputs are the results reported in the reference PDF.

RESULT

Increasing the number of components from k=2 to k=5 improved the raw log-likelihood from **-154.73 to -125.14**. However, the **BIC was lowest for k=3 (356.08)**, indicating that three components are the most appropriate model considering model complexity.